



Politechnika Krakowska
im. Tadeusza Kościuszki

JiPP

Laboratorium II

mgr inż. Krzysztof Szczęśniak

Instrukcja if

Instrukcja **if** w języku C jest jedną z podstawowych konstrukcji sterujących przepływem programu, umożliwiającą wykonywanie fragmentu kodu tylko wtedy, gdy spełniony zostanie określony warunek logiczny. Dzięki temu program może podejmować decyzje i reagować na różne sytuacje.

Instrukcja **if** w swojej podstawowej formie wygląda następująco:

```
if (warunek) {  
    // kod do wykonania, jeśli warunek jest prawdziwy  
}
```

W powyższym przykładzie **warunek to wyrażenie logiczne**, które zwraca wartość **true** (prawda) lub **false** (fałsz). Jeśli warunek jest prawdziwy, to blok kodu znajdujący się w nawiasach klamrowych zostanie wykonany. Jeśli warunek jest fałszywy, kod ten zostanie pominięty.

Przykład:

```
int x = 5;  
if (x > 0) {  
    printf("x jest większe od 0");  
}
```

W tym przykładzie program sprawdza, czy zmienna liczba **jest większa od zera**. Ponieważ warunek **$x > 0$** jest prawdziwy, wyświetlona zostanie wiadomość: „Liczba jest dodatnia”.

Instrukcja if-else

W przypadku, gdy chcemy wykonać jeden fragment kodu, gdy warunek jest prawdziwy, a inny, gdy jest fałszywy, używamy konstrukcji **if-else**.

Instrukcja **if** w swojej podstawowej formie wygląda następująco:

```
if (warunek) {  
    // kod do wykonania, jeśli warunek jest prawdziwy  
}  
else {  
    // kod do wykonania, jeśli warunek jest fałszywy  
}
```

Przykład:

```
int x = 5;  
  
if (x > 0) {  
    printf("x jest większe od 0");  
}  
else {  
    printf("x jest mniejsze lub równe 0");  
}
```

W tym przypadku, ponieważ liczba jest mniejsza od zera, zostanie wyświetlony komunikat: „Liczba jest ujemna lub równa zero”.

Instrukcja else if

Jeżeli chcemy sprawdzić kilka różnych warunków, możemy użyć konstrukcji **else if**. Umożliwia ona wprowadzenie wielu warunków do oceny:

```
if (warunek1) {  
    // kod do wykonania, gdy warunek1 jest prawdziwy  
}  
  
else if (warunek2) {  
    // kod do wykonania, gdy warunek2 jest prawdziwy  
}  
  
else {  
    // kod do wykonania, gdy żaden z warunków nie jest prawdziwy  
}
```

Przykład:

```
int x = 5;  
  
if (x > 0) {  
    printf("x jest dodatnie");  
}  
  
else if (x < 0) {  
    printf("x jest ujemne");  
}  
  
else {  
    printf("x jest równe 0");  
}
```

Program sprawdza kolejno, czy zmienna *x* jest większa od zera, mniejsza od zera, a jeśli żaden z tych warunków nie będzie spełniony, wyświetli wiadomość: „x jest równe 0”.

Zagnieżdżone instrukcje if

Instrukcje if mogą być także zagnieżdżane, co oznacza, że wewnątrz bloku jednego if może znajdować się kolejna instrukcja if. Takie zagnieżdżenie jest przydatne, gdy wymagamy bardziej skomplikowanych decyzji logicznych:

```
if (warunek1) {  
    if (warunek2) {  
        // kod do wykonania, gdy oba warunki są prawdziwe  
    }  
}
```

Przykład:

```
int a = 5, b = 10;  
if (a > 0) {  
    if (b > 0) {  
        printf("Obie liczby są dodatnie\n");  
    }  
}
```

W powyższym przykładzie program sprawdza najpierw, czy *a* jest większe od zera, a następnie, czy *b* również jest dodatnie. Jeśli oba warunki są spełnione, wyświetli komunikat.

Główne operatory logiczne w C

W języku C operatory logiczne służą do wykonywania operacji na wartościach logicznych (prawda/fałsz) oraz do budowania bardziej złożonych warunków, które mogą być wykorzystywane w konstrukcjach takich jak instrukcje warunkowe lub pętle. Operatory te zwracają wartość logiczną: 1 (prawda) lub 0 (fałsz).

1. Operator AND (&&)

Działanie: Zwraca prawdę (1) tylko wtedy, gdy oba operandy są prawdziwe.

Składnia: warunek1 && warunek2

```
int a = 5, b = 10;
if (a > 0 && b > 0) {
    printf("Obie liczby są dodatnie\n");
}
```

2. Operator OR (||)

Działanie: Zwraca prawdę (1), jeśli przynajmniej jeden z operandów jest prawdziwy.

Składnia: warunek1 || warunek2

```
int a = -5, b = 10;
if (a > 0 || b > 0) {
    printf("Przynajmniej jedna liczba jest dodatnia\n");
}
```

Główne operatory logiczne w C

3. Operator NOT (!)

Działanie: Zwraca przeciwną wartość logiczną niż ta, którą operuje. Jeśli warunek jest prawdziwy, zwróci fałsz (0), a jeśli jest fałszywy, zwróci prawdę (1).

Składnia: `!warunek`

```
int a = 5;
if (!(a > 10)) {
    printf("Liczba nie jest większa niż 10\n");
}
```

4. Operatory porównania

- `==` – równość, sprawdza, czy dwie wartości są sobie równe.
- `!=` – nierówność, sprawdza, czy dwie wartości są różne.
- `>` – większe niż.
- `<` – mniejsze niż.
- `>=` – większe lub równe.
- `<=` – mniejsze lub równe.

```
int a = 5, b = 10, c = -3;
if (a > 0 && (b > 0 || c > 0)) {
    printf("a jest dodatnie i przynajmniej jedno z b lub c jest dodatnie\n");
}
```

Tablice jednowymiarowe

Tablice jednowymiarowe są strukturami danych, które umożliwiają przechowywanie wielu wartości tego samego typu pod wspólną nazwą. Tablice ułatwiają organizowanie i zarządzanie zestawami danych, np. liczb, znaków, czy innych typów zmiennych. Każdy element tablicy ma swój indeks, który pozwala na dostęp do konkretnej wartości.

Aby zadeklarować tablicę jednowymiarową, należy określić jej typ oraz rozmiar, czyli liczbę elementów, jakie tablica będzie przechowywać. Składnia wygląda następująco:

```
typ nazwa_tablicy[rozmiar];
```

```
int tablica[5]; // Tablica przechowująca 5 liczb całkowitych
```

```
int tablica[5] = { 1, 2, 3, 4, 5 };
```

Każdy element tablicy ma swój indeks, zaczynający się od 0, a kończący na rozmiar - 1. Aby uzyskać dostęp do konkretnego elementu tablicy, używamy notacji z nawiasami kwadratowymi:

```
int tablica[5] = { 1, 2, 3, 4, 5 };  
printf("%d", tablica[2]); // Wyświetli wartość 3 (element o indeksie 2)
```

Możemy także przypisywać nowe wartości do poszczególnych elementów:

```
int tablica[5] = { 1, 2, 3, 4, 5 };  
tablica[0] = 10; // Zmienia wartość pierwszego elementu na 10
```


Tablice jednowymiarowe

Tablice jednowymiarowe są często wykorzystywane razem z pętlami, które pozwalają na przetwarzanie wszystkich elementów tablicy.

Przykład iteracji przez wszystkie elementy tablicy:

```
int liczby[5] = { 1, 2, 3, 4, 5 };  
for (int i = 0; i < 5; i++) {  
    printf("Element %d: %d\n", i, liczby[i]);  
}
```

Jeżeli tablica zostanie zadeklarowana bez jej jawnej inicjalizacji, elementy tablicy będą miały wartości przypadkowe (zawartość pamięci). Aby tego uniknąć, warto inicjalizować tablicę od razu:

```
int liczby[5] = { 0 }; // Wszystkie elementy są inicjalizowane wartością 0
```

Tablice jednowymiarowe

Podczas pracy z tablicami należy zawsze pamiętać o tym, że **indeksy zaczynają się od 0 i kończą na rozmiar-1**. Próba odwołania się do elementu poza zakresem tablicy (np. liczby[5] w tablicy o 5 elementach) prowadzi do nieokreślonego zachowania programu, które może skutkować błędami lub awarią programu.

Kluczowe informacje:

- Indeksowanie tablicy zaczyna się od 0.
- Tablicę można deklarować i inicjalizować jednocześnie.
- Tablice są bardzo przydatne do przechowywania dużych zbiorów danych i łatwej ich organizacji.

Tablice jednowymiarowe

Przykład praktyczny: obliczanie średniej

Poniżej znajduje się przykład programu, który oblicza średnią z liczb przechowywanych w tablicy:

```
int liczby[5] = { 10, 20, 30, 40, 50 };  
int suma = 0;  
  
// Obliczanie sumy elementów tablicy  
for (int i = 0; i < 5; i++) {  
    suma += liczby[i];  
}  
  
// Obliczanie średniej  
float srednia = suma / 5.0;  
printf("Średnia: %.2f\n", srednia);
```

W tym przykładzie, pętla for przechodzi przez wszystkie elementy tablicy, sumuje je, a następnie na końcu oblicza średnią.

Tablice dwuwymiarowe

Tablice dwuwymiarowe w języku C są rozszerzeniem tablic jednowymiarowych i pozwalają na przechowywanie danych w strukturze przypominającej macierz lub tabelę, czyli w formie wierszy i kolumn. Każdy element tablicy dwuwymiarowej identyfikowany jest przez dwa indeksy: jeden dla wiersza i jeden dla kolumny.

Aby zadeklarować tablicę dwuwymiarową, należy określić jej typ, liczbę wierszy i liczbę kolumn. Składnia wygląda następująco:

```
typ nazwa_tablicy[liczba_wierszy][liczba_kolumn];
```

```
int macierz[3][4]; // Tablica z 3 wierszami i 4 kolumnami, przechowująca liczby całkowite
```

```
int macierz[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```

Tablice dwuwymiarowe

Dostęp do elementów tablicy dwuwymiarowej:

Aby uzyskać dostęp do konkretnego elementu w tablicy dwuwymiarowej, należy podać dwa indeksy: jeden dla wiersza, a drugi dla kolumny:

```
int macierz[3][4] = { {1, 2}, {3, 4}, {5, 6} };  
macierz[1][2] = 10; // Zmienia wartość elementu w 2. wierszu i 3. kolumnie na 10  
printf("%d", macierz[1][2]); // Wyświetli 10
```

Indeksy w tablicach, podobnie jak w tablicach jednowymiarowych, zaczynają się od 0. Zatem `macierz[1][2]` oznacza element znajdujący się w drugim wierszu i trzeciej kolumnie.

Tablice dwuwymiarowe

Przykład iteracji przez wszystkie elementy tablicy:

```
int macierz[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};  
  
for (int i = 0; i < 3; i++) { // Iteracja przez wiersze  
    for (int j = 0; j < 4; j++) { // Iteracja przez kolumny  
        printf("Element [%d][%d]: %d\n", i, j, macierz[i][j]);  
    }  
}
```

W tym przykładzie pętla for iteruje po wszystkich wierszach i kolumnach tablicy, wyświetlając wartości każdego elementu.

Tablice dwuwymiarowe

Przykład praktyczny: macierz liczb i obliczanie sumy elementów

```
// Inicjalizacja tablicy dwuwymiarowej 3x4
int macierz[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

int suma = 0; // Zmienna do przechowywania sumy elementów

// Iteracja przez wszystkie wiersze i kolumny
for (int i = 0; i < 3; i++) { // Iteracja po wierszach
    for (int j = 0; j < 4; j++) { // Iteracja po kolumnach
        suma += macierz[i][j]; // Dodanie wartości elementu do sumy
    }
}

// Wyświetlenie sumy
printf("Suma wszystkich elementów macierzy: %d\n", suma);
```

Opis działania:

- **Inicjalizacja macierzy:** Tablica `macierz[3][4]` została zainicjalizowana wartościami od 1 do 12.
- **Iteracja:** Zagnieżdżone pętle `for` przechodzą przez każdy element macierzy, gdzie zewnętrzna pętla kontroluje wiersze, a wewnętrzna pętla kolumny.
- **Sumowanie elementów:** Każdy element tablicy jest dodawany do zmiennej `suma`, która na początku ma wartość 0.
- **Wyświetlenie wyniku:** Po zakończeniu iteracji wyświetlamy sumę wszystkich elementów macierzy.

Pętle for

Pętla **for** w języku C to jedna z **podstawowych konstrukcji sterujących**, która służy do **wykonywania zestawu instrukcji wielokrotnie**, aż do spełnienia określonego warunku. Pętla **for** jest szczególnie przydatna, gdy znamy z góry liczbę powtórzeń, ponieważ umożliwia połączenie inicjalizacji zmiennych sterujących, warunku zakończenia pętli oraz modyfikacji zmiennych w jednym miejscu.

```
for (inicjalizacja; warunek; modyfikacja) {  
    // kod do wykonania  
}
```

- **inicjalizacja:** Jest to początkowa wartość zmiennej sterującej, np. `int i = 0`.
- **warunek:** Wyrażenie, które jest sprawdzane przed każdą iteracją. Jeśli warunek zwróci wartość `true` (prawdę), pętla się wykona. Gdy warunek będzie fałszywy (`false`), pętla zakończy działanie.
- **modyfikacja:** Jest to wyrażenie, które modyfikuje zmienną sterującą po każdej iteracji (np. zwiększenie lub zmniejszenie wartości zmiennej sterującej).
- **kod do wykonania:** Zestaw instrukcji, który będzie wykonywany w każdej iteracji pętli, jeśli warunek jest prawdziwy.

Pętle for

Przykład:

```
// Pętla for iterująca 10 razy
for (int i = 0; i < 10; i++) {
    printf("Wartość i: %d\n", i);
}
```

Jak działa ten przykład:

1. **Inicjalizacja:** `int i = 0` - zmienna `i` jest ustawiana na wartość początkową 0.
2. **Warunek:** `i < 10` - pętla będzie wykonywać instrukcje, dopóki `i` jest mniejsze niż 10.
3. **Modyfikacja:** `i++` - po każdej iteracji zmienna `i` jest zwiększana o 1.
4. **Instrukcja w pętli:** `printf("Wartość i: %d\n", i);` - wyświetlana jest aktualna wartość `i` w każdej iteracji.

W pętli `for` można zainicjalizować i modyfikować więcej niż jedną zmienną na raz. Na przykład:

```
for (int i = 0, j = 10; i < j; i++, j--) {
    printf("i: %d, j: %d\n", i, j);
}
```

Pętla for bez wszystkich trzech części

Pętla for bez inicjalizacji:

Zmienna sterująca jest zainicjalizowana wcześniej.

```
int i = 0;
for (; i < 10; i++) {
    printf("%d\n", i);
}
```

Pętla for bez warunku:

Pętla stanie się pętlą nieskończoną, dopóki nie użyjemy instrukcji break

```
for (int i = 0; ; i++) {
    if (i == 10) {
        break;
    }
    printf("%d\n", i);
}
```

Pętla for bez modyfikacji:

Zmienna sterująca może być zmieniana w ciele pętli.

```
for (int i = 0; i < 10; ) {
    printf("%d\n", i);
    i++; // modyfikacja w ciele pętli
}
```

Pętla for bez wszystkich trzech części: Możliwa jest nawet taka konstrukcja, gdzie pętla działa nieskończenie.

```
for (;;) {
    printf("Nieskończona pętla\n");
    break; // Aby zakończyć pętlę, używamy break
}
```

Instrukcje break i continue w pętli for

Instrukcja break: Powoduje natychmiastowe zakończenie pętli.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // Pętla zakończy się, gdy i będzie równe 5  
    }  
    printf("%d\n", i);  
}
```

Instrukcja continue: Powoduje przejście do następnej iteracji, pomijając dalszą część ciała pętli.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        continue; // Gdy i wynosi 5, przechodzimy do następnej iteracji  
    }  
    printf("%d\n", i);  
}
```

Pętle while i do-while

Pętle while i do-while w języku C pozwalają na wykonywanie zestawu instrukcji wielokrotnie, dopóki określony warunek jest spełniony. Obie pętle różnią się przede wszystkim tym, kiedy sprawdzany jest warunek.

Pętla while to jedna z najprostszych form pętli w języku C. Jej działanie polega na tym, że **najpierw sprawdzany jest warunek**, a jeśli jest on spełniony (czyli wyrażenie zwraca wartość logiczną true), to pętla wykonuje określony blok kodu. Gdy warunek staje się fałszywy (false), pętla zostaje zakończona.

```
while (warunek) {  
    // kod do wykonania, gdy warunek jest prawdziwy  
}
```

Pętla do-while działa podobnie do while, jednak różni się tym, że **warunek jest sprawdzany po wykonaniu instrukcji w pętli**. Oznacza to, że blok kodu pętli do-while **zawsze zostanie wykonany przynajmniej raz**, nawet jeśli warunek jest fałszywy od samego początku.

```
do {  
    // kod do wykonania  
} while (warunek);
```

Instrukcje sterujące w while i do-while

Instrukcja sterująca **break**:

```
int licznik = 0;

while (1) {
    printf("Licznik: %d\n", licznik);
    licznik++;

    if (licznik == 5) {
        break; // Przerwanie pętli, gdy licznik osiągnie 5
    }
}
```

Taka pętla będzie działać bez przerwy, chyba że zostanie przerwana za pomocą instrukcji **break**. W powyższym przykładzie pętla zostanie przerwana tylko gdy wartość zmiennej **licznik** będzie wynosiła 5.

Instrukcja sterująca **continue**:

```
int licznik = 0;

// Pętla do-while
do {
    licznik++; // Zwiększenie licznika

    // Jeśli liczba jest parzysta, pomiń
    // ją i przejdź do następnej iteracji
    if (licznik % 2 == 0) {
        continue;
    }

    printf("Liczba nieparzysta: %d\n", licznik);
} while (licznik < 9);
```

Warunek **if** sprawdza, czy liczba jest parzysta. Jeśli tak, instrukcja **continue** powoduje, że pętla **pomija bieżącą iterację** i od razu sprawdza warunek **licznik < 9**, aby ewentualnie rozpocząć kolejną iterację

Podobieństwa i różnice między while a do-while

Podobieństwa:

- Obie pętle służą do wielokrotnego wykonywania kodu, dopóki warunek jest spełniony.
- W obu przypadkach można stosować instrukcje sterujące, takie jak **break** (do zakończenia pętli) i **continue** (do pominięcia bieżącej iteracji i przejścia do kolejnej).

Różnice:

- **Sprawdzanie warunku:**
 - W pętli while warunek jest sprawdzany przed wykonaniem kodu, więc jeśli warunek jest fałszywy, pętla może się nie wykonać ani razu.
 - W pętli do-while warunek jest sprawdzany po wykonaniu kodu, więc pętla zawsze wykona się przynajmniej raz.
- **Zastosowanie:**
 - while jest używana, gdy kod może nie zostać wykonany wcale, jeśli warunek nie jest spełniony od początku.
 - do-while jest używana, gdy chcemy, aby kod wykonał się przynajmniej raz, bez względu na wartość początkową warunku

Zadania

Zadanie 1

Napisz program, który pobierze od użytkownika liczbę i sprawdzi, czy jest ona parzysta czy nieparzysta. Wykorzystaj do tego instrukcję if.

Zadanie 2

Napisz program, który pobierze od użytkownika trzy liczby i znajdzie największą z nich, korzystając z instrukcji if.

Zadanie 3

Stwórz program, który wczyta 5 liczb do tablicy jednowymiarowej i znajdzie najmniejszy oraz największy element w tej tablicy.

Zadanie 4

Napisz program, który stworzy tablicę dwuwymiarową 3x3 i obliczy sumę wszystkich elementów tej tablicy. Wartości elementów tablicy mają zostać podane przez użytkownika.

Zadanie 5

Napisz program, który pobierze od użytkownika 10 liczb, zapisze je w tablicy jednowymiarowej, a następnie obliczy ich średnią.

Zadania

Zadanie 6

Napisz program, który pobierze od użytkownika liczbę całkowitą i obliczy sumę jej cyfr, korzystając z pętli while.

Przykład:

Liczba wejściowa: 321

Suma cyfr: $3+2+1=6$

Zadanie 7

Napisz program, który poprosi użytkownika o wprowadzenie liczby większej od 10. Użyj pętli do-while, aby program powtarzał prośbę, dopóki liczba nie spełnia tego warunku.

Zadanie 8

Napisz program, który poprosi użytkownika o wprowadzenie dwóch liczb: wysokości i szerokości prostokąta. Następnie program wyrysuje prostokąt z gwiazdek * o zadanych wymiarach, korzystając z zagnieżdżonych pętli for.